ARMCmp: 比較情報記録の改良に基づくARM32バイナリファジングによるバックドア検出支援

金城 豪志[1,a)]　高田 雄太[2]　熊谷 裕志[2]　神薗 雅紀[2,3]　山内 利宏[4]

概要：IoT 機器を標的とした攻撃には，特定文字列との一致など攻撃者のみが知る条件で悪性挙動を示すコードレベルバックドアがある．この検出には，多数の入力を生成，実行してコードの実行経路を探索するファジングを活用した ROSA などの手法が提案されている．しかし，既存手法は x86-64 環境を前提としており，多くの IoT 機器に採用される ARM32 環境への有効性は明らかではない．そこで ROSA が提供するバックドアデータセット ROSARUM を ARM32 向けに再コンパイルして既存手法を検証した結果，その検出性能の低下を確認した．この要因分析の結果，既存手法が用いるファザー AFL++において，実行時における条件分岐の比較情報を入力変異に利用して条件突破を支援する CmpLog が，ARM32 では有効に機能していなかったことが判明した．本研究では，既存 CmpLog を ARM32 向けに拡張した ARMCmp を提案する．ARMCmp は，比較命令に用いられる値の読み込み幅を起点に，レジスタ間とスタックを介した伝播の追跡により，比較幅を復元する．さらに，ARM32 呼出規約に基づき比較関数の引数を取得する．得られた比較情報を入力変異に活用し，条件突破を支援する．評価において，ARM32 向けに再コンパイルした ROSARUM を用いた結果，ARMCmp により既存手法のバックドア検出数が 233%増加した．

ARMCmp: Enhancing ARM32 Binary Fuzzing for Backdoor Detection through Fine-Grained Comparison Logging

Tsuyoshi Kaneshiro[1,a)]　Yuta Takata[2]　Hiroshi Kumagai[2]　Masaki Kamizono[2,3]
Toshihiro Yamauchi[4]

Abstract: Code-level backdoors in IoT devices trigger malicious behavior only when attacker-known conditions, such as matching specific strings, are satisfied. Fuzzing-based methods such as ROSA detect backdoors by generating and executing inputs to explore paths. However, these methods assume x86-64, and their effectiveness on ARM32, widely used in IoT devices, remains unclear. We recompiled ROSA's ROSARUM backdoor dataset for ARM32 and observed degraded detection performance. Our analysis showed that AFL++'s CmpLog, which guides input mutation using runtime comparison information, did not function effectively on ARM32. We propose ARMCmp, an ARM32 CmpLog extension. ARMCmp restores comparison widths by using load widths as starting points and tracking value propagation through registers and the stack. It also captures arguments to comparison functions according to the ARM32 calling convention. This information guides input mutation to bypass backdoor trigger conditions. We evaluate ARMCmp using the ROSARUM dataset and show that it improves the detection performance of existing methods by 233%.

1. はじめに

IoT 機器を狙った攻撃として，コードレベルバックドア [1] がある．コードレベルバックドアは，特定の条件（以降，トロイトリガー）が満たされた場合にのみ不正な処理

[1] 岡山大学大学院環境生命自然科学研究科
Graduate School of Environmental, Life, Natural Science and Technology, Okayama University
[2] デロイト トーマツ サイバー合同会社
Deloitte Tohmatsu Cyber LLC
[3] 一般社団法人デロイト トーマツ戦略研究所
Deloitte Tohmatsu Strategy Institute
[4] 岡山大学学術研究院環境生命自然科学学域
Faculty of Environmental, Life, Natural Science and Technology, Okayama University
[a)] kaneshiro@s.okayama-u.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

（以降，トロイペイロード）を実行する悪意ある機能である．このようなバックドアの検出に，ROSA をはじめとしたファジングベースバックドア検出手法 [2], [3] が提案されている．ファジングは，未知の脆弱性や動作を発見するための手法である．既存手法は，ファジングを活用して大量の入力を生成する過程で，トロイトリガーを満たす入力を生成，実行時の挙動を観測し，バックドアを検出する．

しかし，既存手法の有効性は主に x86-64 環境で示されており，IoT 機器で広く利用される ARM32 環境における適用可能性は明らかではない．そこで，予備実験として，ROSA の評価に用いられたコードレベルバックドアを含むデータセット ROSARUM [4] を用いて，同一のバックドアを含む x86-64 および ARM32 バイナリを対象に既存手法を適用した．この結果，既存手法は x86-64 バイナリでは検出できたものの，多くの ARM32 バイナリでは検出できないことを確認した．この原因は，ARM32 環境において，既存手法が利用する AFL++がトロイトリガーを満たす入力を生成できないことにある．特に CmpLog が比較条件の突破に必要な情報を適切に収集できないことが判明した．

CmpLog は，実行時の比較情報を入力変異に利用し，文字列や数値比較の突破を支援する機構であり，これらの条件を含むトロイトリガーの突破に重要である．調査の結果，既存 CmpLog には 2 つの課題があることを確認した．1 つ目は，ARM32 では 1，2 バイトの値も 32 ビット汎用レジスタ上で比較されるため，本来の比較幅を取得できず，過大な範囲が入力変異の対象となること，2 つ目は，既存実装は x86-64 の関数呼出規約を前提とするため，ARM32 では比較関数の文字列等を取得できないことであった．

そこで我々は，ARM32 バイナリに対する比較情報収集を改善するため，AFL++ QEMU mode における CmpLog を ARM32 向けに拡張した **ARMCmp** を提案する．ARMCmp は，QEMU translator を用いて比較命令および比較関数の呼び出し地点を検出し，比較幅復元機構および比較関数フック機構を導入することで，比較情報を収集する．比較幅復元機構は，値がメモリから読み込まれた時点の幅と，その後のレジスタ間の伝播およびスタックへの退避・復元を追跡し，比較地点における本来の比較幅を復元する．さらに，命令変換時の追跡だけでは比較幅を特定できない場合に備え，実行時の比較値に基づいてより小さい比較幅の情報を補完的に記録する．比較関数フック機構は，ARM32 における関数呼び出し命令をフックし，ARM32 の関数呼出規約に基づいて引数レジスタを参照することで，`strcmp()`，`memcmp()` などに渡される文字列またはバイト列を記録する．これにより，ARM32 環境で取得が困難であった比較幅や比較関数の引数を CmpLog へ適切に記録し，既存の入力生成処理に活用することで，トロイトリガーのような複雑な文字列比較や数値比較によって保護された条件の突破を支援する．

評価では，ARM32 向けに再コンパイルした ROSARUM を用いて，ARMCmp が既存手法の検出性能をどの程度向上させるかを評価した．評価の結果，ROSARUM synthetic に含まれる 10 件のプログラム中，既存手法では検出できなかった 7 件すべてを ARMCmp では検出できた．

2. 技術背景

2.1 コードレベルバックドア

コードレベルバックドアは，プログラム内部に意図的に埋め込まれた悪意ある機能であり，特定の入力または実行条件が満たされた場合にのみ起動する．バックドアがプログラムへ混入する経路には，2 つのケースがある．

1 つ目は，攻撃者がネットワーク経由で IoT 機器の内部に侵入し，正常なソフトウェアをこれと同じ悪意のあるコードが組み込まれたソフトウェアに差し替える事例である [5]．これにより，通常の動作を装いながら，IoT 機器の内部情報を抜き取られる可能性がある．2 つ目は，ソフトウェアサプライチェーン攻撃 [6] と呼ばれ，製造の工程で，ソフトウェアに不正なコードを混入させる事例である [7], [8]．バックドアは，攻撃者しか知り得ない複雑な条件で起動するよう隠されているため，利用者には認識されず，権限昇格および基盤となるシステムリソースに不正にアクセスし，IoT 機器の内部情報を漏えいする．実際に，PHP, ProFTPD, vsFTPD, および xz などの OSS [9], [10] やネットワークルータのファームウェア [11] にバックドアが組み込まれた事例が報告されている．

2.2 ファジング

2.2.1 概要

ファジングは，検査対象プログラムに多数の入力を与え，クラッシュや異常な挙動を発見する動的テスト手法である．特に，グレーボックスファジングは，実行時のカバレッジ情報等のプログラムの内部構造情報に基づいて入力を選択および変異し，未探索の実行経路を探索する．

代表的なグレーボックスファザーである AFL++ [12] は，検査対象プログラムのコンパイル時にカバレッジ計測用の計装を挿入する．一方，IoT 機器に含まれるソフトウェアでは，ソースコードを入手できず，ARM バイナリのみが検査対象となる場合がある．そこで AFL++では，対象バイナリの動的変換時に計装を挿入する QEMU mode [13] を用いることで，バイナリのみのファジングを可能にする．

2.2.2 AFL++ CmpLog

ファジングの課題の一つに，トロイトリガーをはじめとした比較条件の突破がある．通常の入力変異のみでこのような比較条件を満たすことは，比較対象となる値が複雑になるほど困難になる．この課題に対して，AFL++の CmpLog [14] は，プログラム実行時に観測した比較情報を入力変異に利用することにより，比較条件の突破を支援す

© 2026 Information Processing Society of Japan

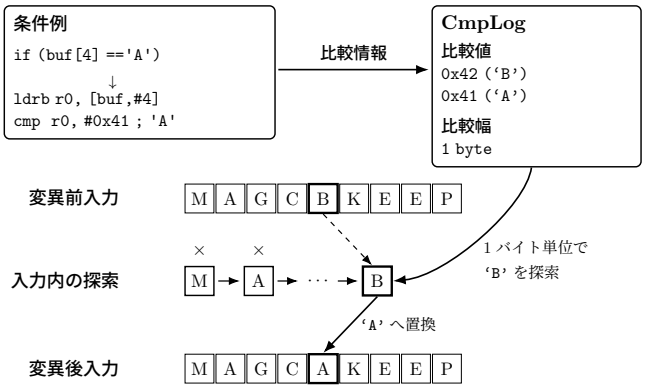


図 1 CmpLog による比較情報の収集と入力変異

る．AFL++ QEMU mode における CmpLog では，対象バイナリを動的に変換する QEMU Translator が，比較命令や比較関数の呼び出しなど，比較情報を収集する箇所を検出する．さらに，検出した箇所に，実行時の比較情報を記録する Runtime helper の呼び出しを挿入する．

比較情報の収集方式は，比較命令に基づく方式と比較関数に基づく方式に大別される．比較命令に基づく方式では，比較に用いられた左右のオペランド値と，比較が行われたバイト数を表す比較幅を記録する．一方，比較関数に基づく方式では，`strcmp()` や `memcmp()` などの引数から，比較対象となる文字列またはバイト列を取得して記録する．

収集した比較情報は，REDQUEEN [14] に基づく入力変異に利用される．**図 1** に，比較命令に基づく方式の動作例を示す．図 1 より，比較情報として入力中の値 ‘`B`’ と比較対象の値 ‘`A`’，および比較幅 `1 byte` が記録される．AFL++は，入力中から ‘`B`’ を 1 バイト単位で探索し，対応する位置を ‘`A`’ へ置換する．このように，比較値は入力へ反映する値を決定し，比較幅は入力を探索および置換するバイト数を決定するために利用される．また，比較関数から取得した文字列やバイト列も，同様に入力変異へ利用され，文字列比較や数値比較の条件突破を支援する．

2.3 ファジングベースバックドア検出手法

2.3.1 概要

ファジングベースのバックドア検出手法として, ROSA [2] と，正常プログラムとの比較に基づく手法（Behavioral Difference Detection Method: BDDM）[3] が提案されている．

ROSA は，検査対象プログラムのみを用いてバックドアを検出する．まず，AFL++ QEMU mode による短時間のファジングで収集した実行時挙動に基づき，類似した処理を受ける入力を入力ファミリとして分類する．その後，長時間のファジングで生成した入力について，同一ファミリ内の他の入力とシステムコール列を比較する．同一ファミリ内で大きく異なる挙動が観測された場合，バックドアが起動した可能性があると判断する．

一方，BDDM は，ファジングで得られた入力を，正常バイナリと検査対象バイナリの両方に再投入し，同一入力に対するシステムコール列を比較する．検査対象バイナリでのみファイル操作，外部通信，コマンド実行などが観測された場合，バックドアが起動した可能性があると判断する．

このようにファジングベースのバックドア検出手法は，ファジングによって多数の入力を生成し，入力実行時のシステムコール列に現れる通常動作との差異から，バックドアの起動を検出する点で共通している．

2.3.2 既存手法におけるファジングの役割

ROSA および BDDM では，ファジングによってトロイトリガーを満たす入力を生成し，トロイペイロードの実行に伴う挙動を観測することでバックドアを検出する．したがって，ファザーがトロイトリガーを突破できない場合，バックドアの挙動を観測できず，検出できない．

トロイトリガーの多くは，特定の文字列や数値との一致を要求する．このため，トリガー入力の生成は，2.2.2 項で述べた CmpLog が重要となる．そこで，CmpLog の寄与を確認するため，x86-64 環境において，バックドアデータセット ROSARUM synthetic [4] の 10 件に CmpLog を無効化した AFL++を適用し，トロイトリガーの突破可否を確認した．確認の結果，CmpLog の有効化で突破できた 10 件のうち 7 件は，CmpLog の無効化で突破できなかった．したがって，文字列や数値との比較を含むトロイトリガーを突破し，バックドアを起動するためには，CmpLog が重要な役割を果たすことが分かる．

3. 予備実験

3.1 ARM32 における既存手法の適用

ファジングベースバックドア検出手法は，主に x86-64 環境を対象として有効性が示されている．一方，多くの IoT 機器で採用される ARM32 環境における有効性は明らかにされていない．そこで本節では，予備実験として，既存手法を ARM32 環境へ適用し，x86-64 環境と同様にバックドアを検出できるかを検証する．本実験では ROSARUM の各プログラムをベースに，x86-64 バイナリおよび ARM32 バイナリを用意し，両バイナリに対して既存手法を適用した．なお，実行環境の詳細は後続の 5 章で述べる．

表 1 に，x86-64 環境および ARM32 環境における既存手法のバックドア検出結果を示す．x86-64 環境では，ROSA および BDDM のいずれも ROSARUM synthetic 10 件すべてを検出できた．一方，ARM32 環境で検出できたのは，いずれの手法も 3 件にとどまった．この結果から，x86-64 環境と比較して ARM32 環境では，既存手法のバックドア検出性能が低下することがわかる．

この要因について ARM32 環境における未検出バイナリのファジング過程を調査した結果，トロイトリガーを満たす入力が生成されていなかったため，トロイペイロードの

© 2026 Information Processing Society of Japan

表 1　x86-64 および ARM32 環境における既存手法のバックドア検出結果（検出回数 / 試行回数）

Target	ROSA		BDDM	
	x86-64	ARM32	x86-64	ARM32
sudo	10/10	0/10	10/10	0/10
libpng	10/10	0/10	10/10	0/10
libsndfile	10/10	0/10	10/10	0/10
libtiff	10/10	10/10	10/10	10/10
libxml2	10/10	0/10	10/10	0/10
Lua	10/10	10/10	10/10	10/10
OpenSSL	10/10	0/10	10/10	0/10
PHP / unserialize	10/10	10/10	10/10	10/10
Poppler	10/10	0/10	10/10	0/10
SQLite3	10/10	0/10	10/10	0/10

実行時挙動に基づく既存手法はバックドアを検出できなかった．さらに，ARM32 環境における未検出プログラムと x86-64 環境で CmpLog を無効化した場合にトロイトリガーを突破できなかったプログラムは，Lua，SQLite3 を除いて一致した．このため，CmpLog が ARM32 においても重要であることがわかる．次節では本要因を明らかにするため，既存 CmpLog の実装を調査した結果を述べる．

3.2　ARM32 における既存 CmpLog の課題

AFL++ QEMU mode における CmpLog は，対象バイナリの実行中に比較情報を収集するため，対象アーキテクチャの命令体系，QEMU における命令変換処理，および関数呼出規約に依存する．本実装を調査した結果，ARM32 環境における既存 CmpLog には，二つの課題があった．

3.2.1　課題 1：比較命令に基づく方式における比較幅

2.2.2 項に示した通り，比較幅は，後続の入力生成で入力を探索および置換する単位を決定する．したがって，比較条件に対応する入力領域を適切に変異するには，比較幅を正しく記録する必要がある．

x86-64 環境では，`al`，`ax`，`eax` などの部分レジスタや，`cmpb`，`cmpw` などの命令形式から比較幅が判別できる．このため，CmpLog は比較命令の解析により，1 バイト，2 バイト，または 4 バイトのどの幅で比較が行われたかを比較的直接取得できる．一方，ARM32 環境では，`ldrb` や `ldrh` によって読み込まれた 1，2 バイトの値も，後続の演算や比較では 32 ビット汎用レジスタ上で扱われる．このため，既存の ARM32 向け CmpLog では，比較命令のオペランドが 32 ビットレジスタであることに基づき，本来は 1，または 2 バイトの比較であっても，4 バイト比較として記録される場合がある．すなわち，x86-64 環境のように比較命令そのものから比較幅を直接取得することが難しく，既存実装では比較幅が実質的に 32 ビットへ固定されてしまう．

比較幅が実際よりも大きく記録される場合，比較条件に対応する入力領域だけでなく，隣接するバイトまで変異の対象となる．この結果，比較対象の値を入力へ反映できても，無関係な入力領域を破壊し，後続の入力検証や条件突破を妨げる可能性がある．

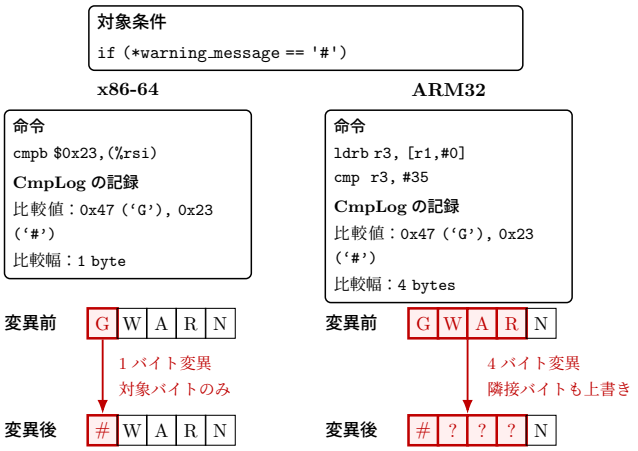


図 2　比較幅の記録が入力変異に与える影響

図 2 に，libpng の条件 `if (*warning_message == '#')` を例として，比較幅の違いが入力変異に与える影響を示す．x86-64 環境では，この条件が 1 バイト比較命令 `cmpb $0x23,(%rsi)` として現れるため，CmpLog は比較値（`0x47`，`0x23`）と比較幅 1 バイトを記録する．その結果，入力 `GWARN` の先頭 1 バイトのみが置換され，`#WARN` が生成される．一方，ARM32 環境では，`ldrb r3, [r1,#0]` で読み込まれた 1 バイト値が，`cmp r3, #35` によって 32 ビットレジスタ上で比較される．このため，既存 CmpLog は本来のロード幅を判別できず，比較幅を 4 バイトで記録する．結果，先頭バイトだけでなく隣接する 3 バイトも変異対象となり，無関係な入力領域が上書きされてしまう．したがって，ARM32 環境で適切な比較幅を記録するには，比較値が由来するロード命令の幅を追跡する必要がある．

3.2.2　課題 2：比較関数に基づく方式における引数取得

比較関数に基づく方式は，`strcmp()` や `memcmp()` などの呼び出し時に，比較対象となる引数を取得して記録する．既存の x86-64 向け実装では，x86-64 の関数呼出規約に基づき，引数レジスタから比較対象となるポインタを取得する．一方，既存の ARM32 向け実装には，これらの関数呼び出し地点で，ARM32 の関数呼出規約に基づいて引数を取得する処理が存在しなかった．このため，文字列比較等に用いられた値が CmpLog へ記録されず，後続の入力生成に比較対象の文字列やバイト列を利用できなかった．

以上より，ARM32 環境では，比較幅が実際より粗く記録されているとともに，比較対象の引数自体も取得できていない．このため，トロイトリガーの比較情報が AFL++ の入力生成へ十分に反映されていなかったと考えられる．

4.　ARMCmp

4.1　設計方針

本研究の目的は，ARM32 バイナリの実行中に観測され

© 2026 Information Processing Society of Japan

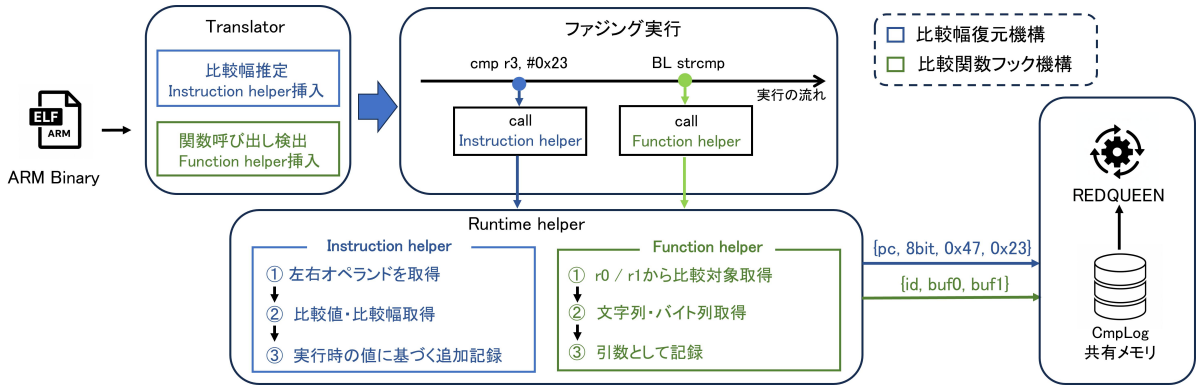


図 3　ARMCmp の全体像

る比較情報を，AFL++の入力生成に利用可能な形で記録し，ARM32 環境の CmpLog を有効に機能させ，既存手法の検出性能を向上させることである．本研究では，入力生成アルゴリズム自体は変更せず，その前段階である比較情報の収集処理を ARM32 向けに拡張した ARMCmp を提案する．なお，修正量は 6 ファイルで 432 行である．

図 3 に ARMCmp の全体像を示す．ARMCmp は，3.2 節に示した 2 つの課題に対処するための 2 つの機構から構成される．1 つ目の課題の対処として，本来の比較幅を復元する比較幅復元機構を提案する．比較幅復元機構では，値がメモリから読み込まれた際の幅，レジスタ間の伝播，およびスタックへの退避・復元を追跡することで，比較地点における本来の比較幅を可能な範囲で復元する．これにより，本来は 1 バイトまたは 2 バイトである比較を，一律に 4 バイト比較として記録することを避ける．

2 つ目の課題の対処として，ARM32 の関数呼出規約に基づいて比較対象を取得する比較関数フック機構を提案する．比較関数フック機構は関数呼び出し地点を検出し，第 1 引数および第 2 引数が格納されるレジスタから，`strcmp()`，`strncmp()`，`memcmp()` などに渡される比較対象のポインタを取得する．取得した文字列またはバイト列を CmpLog へ記録することで，文字列比較やメモリ比較に用いられた値を，後続の入力生成へ反映できるようにする．

4.2　比較幅復元機構

比較幅復元機構は，比較命令に現れるレジスタ幅ではなく，比較値が由来するデータの幅を推定し，この幅に基づいて比較情報を記録する機構である．本機構は，QEMU Translator による幅情報の追跡と，Runtime helper による比較情報の記録から構成される．

4.2.1　Translator：幅情報の追跡

QEMU Translator は，対象バイナリの命令列を変換する際に，ARM32 の各レジスタが保持する値の由来幅を追跡する．Translator の状態にレジスタ番号を添字とする幅情報配列を設け，各レジスタの値が 8 ビット，16 ビット，

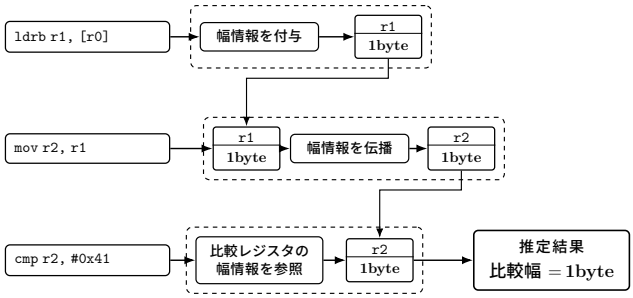


図 4　幅情報の伝播に基づく比較幅の復元

または 32 ビットのいずれに由来するかを記録する．

図 4 に幅情報の追跡例を示す．`ldrb` によって 1 バイトの値がレジスタ `r1` へ読み込まれると，`r1` に 1 バイト由来を示す幅情報を付与する．続いて，`mov` によって値が `r2` へ移動すると，幅情報も `r2` へ伝播する．その後，`cmp` 命令において `r2` の幅情報を参照することで，レジスタ幅である 32 ビットではなく，本来の比較幅を 1 バイトと判定する．一方，シフト，回転，複数値の結合，および元の値幅を安全に保持できない算術演算では，誤った推定を避けるため，出力レジスタの幅情報を 32 ビットへ戻す．また，比較値が比較前にスタックへ退避されると，レジスタに関連付けていた幅情報のみでは，その値が再びレジスタへ読み込まれた際に元のデータ幅を復元できない．そこで，スタックポインタまたはフレームポインタとオフセットの組に幅情報を関連付け，値の保存時に幅情報を記録し，読込み時に読込み先レジスタへ復元する．したがって，本機構は，安全に伝播できる場合に限って狭幅情報を保持する保守的な値幅追跡である．

比較地点では，Translator が比較オペランドに対応する幅情報を配列から取得し，比較幅を保守的に決定する．両オペランドが 8 ビット由来であれば 8 ビット比較，両者が 16 ビット以下であれば 16 ビット比較として扱い，それ以外の場合は 32 ビット比較として扱う．この後，決定した比較幅に対応する helper（以降，Instruction helper）の呼び出しを QEMU が生成する変換コードへ挿入する．

© 2026 Information Processing Society of Japan

4.2.2 Runtime helper：比較情報の記録

ファジング実行時，比較地点へ到達すると，Translator が挿入した Instruction helper が呼び出される．Instruction helper は，比較地点，比較幅，および左右のオペランド値を {pc, width, lhs, rhs} として CmpLog 共有メモリへ記録する．さらに，Runtime helper では，実行時の比較値に基づいて比較幅を補完する．実行時のオペランド値が符号拡張またはゼロ拡張によって 16 ビット値または 8 ビット値を拡張したものと判定できる場合には，対応する比較幅の情報も追加で記録する．これにより，Translator 側で比較幅を判定できなかった場合にも，実行時の値に基づいて比較幅を補完し，後続の入力変異へ反映できる．

4.3 比較関数フック機構

比較関数に基づく方式では，`strcmp()` や `memcmp()` へ渡された比較対象を記録する必要がある．しかし，既存実装には，ARM32 の関数呼出規約に対応した引数取得処理が存在しない．そこで，比較関数フック機構を導入した．

4.3.1 Translator：関数呼び出し地点の検出

比較関数フック機構は，ARM32 の手続き呼出規約である AAPCS32 [15] をもとに，Translator の対象バイナリの命令列を変換する過程で `BL` および `BLX` の関数呼び出し命令を検出する．この際に，比較関数候補に限定せず，すべての BL/BLX 命令に helper を挿入する．関数呼び出し命令を検出すると，元の関数呼び出し処理を生成する直前に，比較関数用の CmpLog helper（以降，Function helper）を呼び出す処理を QEMU 変換コード中に生成する．

4.3.2 Runtime helper：比較対象の記録

ファジング時に Function helper が呼び出されると，第 1 および第 2 引数が格納された `r0` と `r1` の値を，比較対象候補を指すアドレスとして取得する．本機構は比較関数に限定せず全関数呼び出しをフックするため，各アドレスが指すメモリ領域を安全に読み取れるかを確認する．両方の領域を読み取れる場合に，それぞれから最大 31 バイトを取得し，CmpLog 共有メモリの関数比較用領域へ記録する．

ここで，取得した比較情報は，{`id`, `buf0`, `buf1`} として表される．`id` は関数呼び出し地点を識別する値，`buf0` および `buf1` は各引数が指すデータを表す．取得したデータがヌル終端文字列である場合には，終端位置に基づく長さを記録し，それ以外の場合には固定長のバイト列として記録する．例えば，`strcmp(input, "AUTH")` の呼び出しでは，`r0` と `r1` がそれぞれ `input` と `"AUTH"` の先頭アドレスを保持するため，両者が指す文字列を取得して記録する．一方，`malloc(16)` のように引数がポインタではない場合や，いずれかの引数がヌルポインタまたは未割当て領域を示す場合には，メモリ領域を安全に読み取れないため記録しない．これにより，比較関数で使用された文字列やバイト列のみを，後続の入力生成へ利用できるようにする．

表 2 評価に用いる構成

構成	比較幅復元機構	比較関数フック機構
`Baseline`	無効	無効
`Ins-only`	有効	無効
`Func-only`	無効	有効
`ARMCmp`	有効	有効

5. 評価

ARMCmp が ARM32 環境における CmpLog の比較情報収集を改善し，条件突破可能になったか，およびファジングベースのバックドア検出手法の性能に寄与したかについて，以下の Research Questions (RQs) に取り組む．

- RQ1: ARMCmp による ARM32 バイナリファジングのコードカバレッジの向上，およびそのオーバヘッドはどの程度か？
- RQ2: ARMCmp は，ファジングベースバックドア検出手法の検出性能をどの程度改善するか？

5.1 評価環境

RQs 1, 2 における実験は，Ubuntu 24.04.2 LTS を搭載した同一構成の計算機上で実行する．計算機は Intel Xeon Gold 5120 CPU および 24.0GB のメモリを備える．また，既存手法が利用するファザーには AFL++ 4.36a を用いる．

RQs 1, 2 の評価対象として ROSARUM [4] を用いる．ROSARUM は，バックドア検出ツールの評価を目的としたデータセットであり，多様なプログラムにおけるバックドア検出能力を測定できる．ROSARUM は 7 件の実在バックドア（authentic）と 10 件の合成バックドア（synthetic）を含んでいる．本研究では，この中でもライブラリをはじめとしたさまざまなプログラムを収録する synthetic に適用し，ARMCmp の有効性について評価する．

また，ARMCmp の各機構による貢献を評価するため，**表 2** に示す各構成を評価する．`Baseline` は既存 CmpLog を用いた AFL++ QEMU mode であり，ベースラインとして用いる．`Ins-only`，`Func-only`，および `ARMCmp` は，ARMCmp を組み込んだ AFL++ QEMU mode を用いる構成である．`Ins-only` は，比較幅復元機構のみを有効化した構成，`Func-only` は，比較関数フック機構のみを有効化した構成である．`ARMCmp` は，両機構を有効化した提案手法による構成である．

5.2 RQ1: ファジング性能への影響

方法: ARMCmp が ARM32 バイナリファジングの探索性能に与える影響を，コードカバレッジと実行時オーバヘッドの観点から評価する．表 2 に示す各構成について，同一の初期シードおよび設定を用い，ROSARUM の各対象に対して 10 時間のファジングを 5 回ずつ実施する．なお，各試行は単一のファザーインスタンスで実行する．

© 2026 Information Processing Society of Japan

コードカバレッジは，各構成の各試行で，ファジング中に新しい条件分岐へ到達した時に保存される入力集合を再実行し，gcov [16] を用いて各条件分岐の到達状況を測定することにより評価する．評価指標は以下の 3 つである．

- **到達条件数**: 真，または偽側の少なくとも一方に到達した条件数．
- **両分岐到達条件数**: 真，偽側両方に到達した条件数．
- **新規両分岐条件数**: Baseline では 5 試行すべてで両分岐に到達せず，ARMCmp では 5 試行中 3 試行以上で両分岐に到達した条件数．

到達条件数は探索範囲，両分岐到達条件数は比較条件の突破性能を評価するために用いる．また，新規両分岐条件数は，ARMCmp によって新たに突破可能となった条件を抽出し，この特徴を分析するために用いる．

さらに，各構成の実行時オーバヘッドを評価する．通常のファジング実行時間には入力生成や変異の処理も含まれる．このため，ARMCmp による QEMU 実行部分の影響を切り分ける目的で CmpLog を有効にした afl-qemu-trace を用いる．同一の入力集合 (1,000 件) を各構成で 5 回実行し，各入力の実行時間の中央値を求め，さらに対象ごとの中央値を求めることで算出した．これにより，入力ごとのばらつきやタイムアウトの影響を抑えた．

結果: 表 3 に，`Baseline` に対する `ARMCmp` の条件カバレッジ差分を示す．各差分は，`ARMCmp` から `Baseline` の結果を引いた値である．各構成のより詳細な結果は付録 **表 A·1** に示す．`ARMCmp` は，10 件中 5 件で到達条件数，6 件で両分岐到達条件数を増加させた．特に sudo では，従来突破できなかった認証条件 `strcmp(pass, "let_me_in")` を突破し，以降の処理へ到達したことで，到達条件数が 608 件増加した．`Func-only` でも同様の改善が確認され，比較関数フック機構が文字列比較の突破に寄与したと考えられる．一方，OpenSSL は，条件カバレッジの差分を確認できなかった．この原因として，OpenSSL は，`strcmp()` や `memcmp()` といった明示的な比較関数や数値比較が少ないため，ARMCmp が効果的に機能しなかったことが挙げられる．また，新規両分岐条件では，libpng では `strcmp()` による文字列比較，SQLite3 では `sqlite3_strnicmp()` による文字列比較や識別子判定などが確認された．オーバヘッドは，全対象で実行時間の増加を確認し，最も増加した対象は Poppler で，約 20%増加した．一方，libxml2 や libpng の増加率はそれぞれ 3.1%，1.9%にとどまり，`Baseline` と同程度であった．詳細は付録の **表 A·2** に示す．

RQ1 への回答: ARMCmp は，比較情報の収集処理により最大で約 20%のオーバヘッドを伴う一方，文字列比較，数値比較，構文解析等に関する複雑な条件を効率的に突破でき，両分岐到達数の増加に貢献する．このことから，ARMCmp の適用には，実行時オーバヘッドと探索性能向上のトレードオフがある．

表 3 `Baseline` に対する `ARMCmp` の条件カバレッジ差分

対象	Δ 到達条件	Δ 両分岐条件	新規両分岐条件
sudo	+608	+100	100
libpng	−8	−11	2
libsndfile	+21	+3	8
libtiff	−8	+8	1
libxml2	+152	+29	28
Lua	−6	−18	0
OpenSSL	0	0	0
PHP/unserialize	+183	+37	35
Poppler	−1	−50	12
SQLite3	+132	+72	30

表 4 ARMCmp を組み込んだ ROSA および BDDM におけるバックドア検出回数（検出回数 / 試行回数）

対象	ROSA		BDDM	
	Baseline	ARMCmp	Baseline	ARMCmp
sudo	0/10	10/10	0/10	10/10
libpng	0/10	10/10	0/10	10/10
libsndfile	0/10	10/10	0/10	10/10
libtiff	10/10	10/10	10/10	10/10
libxml2	0/10	10/10	0/10	10/10
Lua	10/10	10/10	10/10	10/10
OpenSSL	0/10	10/10	0/10	10/10
PHP/unserialize	10/10	10/10	10/10	10/10
Poppler	0/10	10/10	0/10	10/10
SQLite3	0/10	10/10	0/10	10/10

5.3 RQ2：バックドア検出手法の検出性能への寄与

方法: ARMCmp が，ファジングベースバックドア検出手法による検出結果に与える影響を評価する．具体的には，ROSARUM synthetic の各プログラムに対して，ARMCmp を組み込んだ ROSA と BDDM をそれぞれ 10 回適用し，バックドアを検出できるか評価する．なお，実験条件は既存手法の評価方法に合わせ，各試行を 8 時間，6 つのファザーインスタンスで実施する．

結果: 表 4 に ROSA および BDDM によるバックドア検出結果を示す．`Baseline` を用いた場合，`libtiff`，`Lua`，`PHP/unserialize` の合計 3 件のみが検出されたが，`ARMCmp` を用いた場合は，ROSA および BDDM のいずれにおいても，全対象，全試行でバックドアを検出できた．

RQ2 への回答: ARMCmp は，ファジングベースバックドア検出手法によるバックドア検出数 233%増加 (3 件から 10 件へ) に貢献した．したがって，ARMCmp による CmpLog の改善は，従来生成が難しかったトロイトリガー入力の生成を可能にし，ARM32 環境におけるファジングベースバックドア検出手法等のファジングを応用する手法の性能向上に貢献する．

6. 議論

ARMCmp の基本構造は，QEMU Translator で比較地

© 2026 Information Processing Society of Japan

点を検出し，変換後コードへ helper 呼び出しを挿入し，実行時に runtime helper が CmpLog へ比較情報を記録する点にある．この構造自体は ARM32 に限定されず，QEMU mode でサポートされる他アーキテクチャにも適用可能である．一方で，具体的な比較幅推定と比較関数引数の取得方法は，各アーキテクチャの命令セットおよび ABI に依存する．例えば x86-64 では，命令のオペランド幅から比較幅を直接取得しやすい．これに対し，ARM32 や MIPS のような RISC 系アーキテクチャでは，ロード時の幅と比較時のレジスタ幅が分離して現れる場合があるため，値がどの幅で読み込まれたかを追跡する設計が有効となる．また，比較関数に基づく収集では，ARM32 では `r0`/`r1` を参照したが，MIPS では `a0`/`a1` のように，各 ABI に対応する引数位置を取得する必要がある．したがって，本研究の実装は ARM32 に特化した規則を含むものの，Translator 側で比較地点を検出し，runtime helper 側で比較情報を記録するという設計は，他アーキテクチャにも応用可能である．

実際に，ARMCmp の実装方式を MIPS 向けに応用した CmpLog（MIPSCmp）を用いたところ，ROSA および BDDM では従来検出できなかった synthetic プログラム 7 件が検出可能になった．詳細は付録 A.3 に示す．

7. 関連研究

バイナリファジングによる複雑な文字列や数値比較といった困難な条件を突破する手法として，動的バイナリ解析やテイント解析を用いた手法が提案されている．STEELIX [17] は，動的バイナリ計装でプログラム状態を観測し，マジックバイトを満たす入力変異を支援する．VUzzer [18] は，静的・動的解析によって入力中の重要なバイトを特定し，優先的に変異させる．TaintScope [19] は，動的テイント解析により checksum などの入力検証を特定し，支援する．

本研究は，既存 CmpLog における比較情報の収集を ARM32 向けに改善することで，AFL++の入力生成を支援する．既存手法が条件突破のための解析や入力変異手法の高度化に着目する一方で，提案手法は入力変異に必要な情報を拡充させる点で手法が異なる．すなわち，両者は補完的な関係にあり，組み合わせることで，より困難な条件に対する探索性能の向上が期待できる．

8. おわりに

本研究では，ARM32 環境におけるファジングベースバックドア検出の性能向上を目的として，ファザー AFL++の CmpLog を ARM32 上で有効に機能させる ARMCmp を提案した．ARMCmp は，メモリロード時のデータ幅やレジスタ・スタックを介した値の伝播を追跡することで比較命令の比較幅を復元するとともに，ARM32 の関数呼出規約に基づいて比較関数の引数を取得する．これにより，文字列や数値との比較に必要な情報を収集し，後続の入力生成へ利用できるようにした．

ARM32 向け ROSARUM を用いた評価では，既存 CmpLog では突破できなかった条件への到達を改善し，ROSA および BDDM で未検出の 7 件のバックドアを検出できた．ARM32 以外の他アーキテクチャにも提案手法を拡張・応用し，その有効性を示すことは今後の課題である．

謝辞 本研究は，JST 国家戦略分野の若手研究者及び博士後期課程学生の育成事業（博士後期課程学生支援）JPMJBS2403 の支援を受けたものです．

参考文献

[1] Thomas, S. L. and Francillon, A.: Backdoors: Definition, Deniability and Detection, *RAID* (2018).
[2] Kokkonis, D. et al.: ROSA: Finding Backdoors with Fuzzing, *ICSE* (2025).
[3] 金城豪志，高田雄太，熊谷裕志ほか：バイナリファジングを利用したトロイ化 IoT 機器の検出手法，情報処理学会論文誌， Vol. 67, No. 2, pp. 436–450 (2026).
[4] BINSEC: ROSARUM: A Novel Backdoor Detection Benchmark, `https://github.com/binsec/rosarum` (accessed 2026-08-07).
[5] Noman, H. A. and Abu-Sharkh, O. M. F.: Code Injection Attacks in Wireless-Based Internet of Things (IoT): A Comprehensive Review and Practical Implementations, *Sensors*, Vol. 23, No. 13, p. 6067 (2023).
[6] Williams, L. et al.: Research Directions in Software Supply Chain Security, *ACM Trans. Softw. Eng. Methodol.*, Vol. 34, No. 5 (2025).
[7] Ladisa, P. et al.: SoK: Taxonomy of Attacks on Open-Source Software Supply Chains, *IEEE S&P* (2023).
[8] Ohm, M. et al.: Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks, *DIMVA* (2020).
[9] Schuster, F. and Holz, T.: Towards Reducing the Attack Surface of Software Backdoors, *ACM CCS* (2013).
[10] National Vulnerability Database: CVE-2024-3094 Detail, `https://nvd.nist.gov/vuln/detail/CVE-2024-3094` (accessed 2026-08-20).
[11] Lee, M.: D-Link Routers Found to Contain Backdoor, `https://www.zdnet.com/article/d-link-routers-found-to-contain-backdoor` (accessed 2024-05-22).
[12] Fioraldi, A. et al.: AFL++: Combining Incremental Steps of Fuzzing Research, *USENIX WOOT* (2020).
[13] AFLplusplus: QEMU-AFL (qemuafl), `https://github.com/AFLplusplus/qemuafl` (accessed 2026-06-25).
[14] Aschermann, C. et al.: REDQUEEN: Fuzzing with Input-to-State Correspondence, *NDSS* (2019).
[15] Arm Limited: Procedure Call Standard for the Arm Architecture (AAPCS32), Version 2025Q4, `https://github.com/ARM-software/abi-aa/releases/download/2025Q4/aapcs32.pdf` (accessed 2026-06-25).
[16] GNU Project: gcov—a Test Coverage Program, `https://gcc.gnu.org/onlinedocs/gcc/Gcov.html` (accessed 2026-06-25).
[17] Li, Y. et al.: STEELIX: Program-State Based Binary Fuzzing, *ESEC/FSE* (2017).
[18] Rawat, S. et al.: VUzzer: Application-Aware Evolutionary Fuzzing, *NDSS* (2017).
[19] Wang, T. et al.: TaintScope: A Checksum-Aware Directed Fuzzing Tool for Automatic Software Vulnerability Detection, *IEEE S&P* (2010).

© 2026 Information Processing Society of Japan

付　　録

A.1　各構成ごとの条件カバレッジ

表 A·1 に，RQ1 で評価した各構成の条件カバレッジの詳細を示す．表 A·1 では，Baseline と ARMCmp のほかに，Ins-only および Func-only の結果も含めている．なお，新規両分岐条件数は，Baseline を基準に算出するため，Baseline の値は N/A とした．

結果より，Ins-only，Func-only，ARMCmp はそれぞれ到達条件数で 5 件，4 件，5 件，両分岐到達条件数で 4 件，7 件，6 件，新規両分岐条件数で 8 件，6 件，8 件の対象において Baseline を上回った．

また，Ins-only では PHP/unserialize で 129 件の最多の新規両分岐条件が確認され，`if(yych == 'N')` や `if(yych <= '/')` のように，入力文字を直接判定する字句解析中の条件が新たに突破できていた．また，libxml2 でも `if ((RAW == 'n') && (NXT(1) == 'o'))` のような数値条件を突破可能になっていた．このことから，Ins-only は入力バイトや整数値に対する細粒度な比較条件の突破に寄与していると考えられる．

一方，Func-only では，sudo で 100 件の最大の新規両分岐条件が確認され，`if (strcmp(pass, "let_me_in") == 0)` をはじめとした文字列比較関数が突破できるようになり，libxml2 についても `!strcmp(handler->name, "UTF-16BE")` を突破可能になっていた．このことから，Func-only は文字列比較関数やライブラリ関数による比較が突破できることにより，この後続処理を含む条件突破を可能にしていると考えられる．

A.2　各構成のオーバヘッド

表 A·2 に，各構成における同一入力の実行時間，および Baseline に対する増減率を示す．表 A·2 より，いずれの構成においても，実行時間は Baseline と最大でも約 20% の増加にとどまり，ARMCmp の比較情報収集処理による極端な実行時オーバヘッドが生じないことを確認した．一方，対象によってオーバヘッドの大きさには差があり，最も増加したものは，Poppler であり，約 0.0742 秒であり，約 20%の増加であった．この要因として，Poppler では，ARMCmp が 1 入力あたりに収集する比較情報量が Baseline の約 2 倍となっており，比較情報の記録処理が増加したことが考えられる．これに対して，libpng，libtiff，libxml2 における ARMCmp の実行時間は，Baseline と近く，両構成の実行時間に大きな差は確認されなかった．

以上より，ARMCmp の実行時オーバヘッドは，対象プログラムの比較命令・比較関数の数およびそれらから収集する比較情報量に依存する．また，比較情報の収集量が増加することで，一定の実行時間増加が生じる場合がある一方で，従来 CmpLog では突破できなかった条件への到達を可能にしている．したがって，ARMCmp のオーバヘッドの増加は，探索性能の向上とのトレードオフであるといえる．

A.3　提案手法の MIPS アーキテクチャへの応用

6 章で示したアーキテクチャの応用可能性について，実際に ARMCmp の実装方式を MIPS に応用した MIPSCmp を用いて，MIPS 環境における有効性を簡易的に検証した．実験環境は 5.1 節に示した通りであり，MIPS にクロスコンパイルした ROSARUM synthetic に MIPSCmp を組み込んだ ROSA および BDDM をそれぞれ 1 回ずつ適用した．その結果を**表 A·3** に示す．表 A·3 より，Baseline で検出できなかった 7 件のプログラムすべてが，MIPSCmp を組み込んだことによって検出可能になった．このことから，ARMCmp の実装方式は，MIPS をはじめとした他アーキテクチャに対して，応用可能であると言える．

A.4　提案手法の再現性

提案手法の再現性および研究コミュニティの活性化を目的に，ARMCmp による拡張コードは AFL++ のコードリポジトリへのプルリクエストを通じて，査読付き論文として採録後に公開する予定である．

© 2026 Information Processing Society of Japan

表 A·1 各構成における条件カバレッジ

対象	構成	到達条件	両分岐条件	新規両分岐条件	対象	構成	到達条件	両分岐条件	新規両分岐条件
sudo	Baseline	1,632	224	N/A	Lua	Baseline	2,149	1,727	N/A
	Ins-only	1,632	224	0		Ins-only	2,153	1,740	8
	Func-only	2,240	324	100		Func-only	2,118	1,680	0
	ARMCmp	2,240	324	100		ARMCmp	2,143	1,709	0
libpng	Baseline	1,067	519	N/A	OpenSSL	Baseline	1,009	396	N/A
	Ins-only	1,058	505	1		Ins-only	1,009	396	0
	Func-only	1,058	511	0		Func-only	1,009	397	0
	ARMCmp	1,059	508	2		ARMCmp	1,009	396	0
libsndfile	Baseline	1,617	859	N/A	PHP/unserialize	Baseline	4,821	1,915	N/A
	Ins-only	1,599	855	2		Ins-only	4,904	1,909	129
	Func-only	1,613	861	3		Func-only	4,778	1,952	39
	ARMCmp	1,638	862	8		ARMCmp	5,004	1,952	35
libtiff	Baseline	2,077	1,039	N/A	Poppler	Baseline	6,602	3,648	N/A
	Ins-only	2,022	1,016	5		Ins-only	6,610	3,747	11
	Func-only	2,057	1,040	0		Func-only	6,620	3,662	53
	ARMCmp	2,069	1,047	1		ARMCmp	6,601	3,598	12
libxml2	Baseline	5,438	2,337	N/A	SQLite3	Baseline	8,040	2,388	N/A
	Ins-only	5,524	2,338	12		Ins-only	8,367	2,463	11
	Func-only	5,548	2,262	29		Func-only	8,259	2,444	25
	ARMCmp	5,590	2,366	28		ARMCmp	8,172	2,460	30

表 A·2 各構成における同一入力の実行時間 (ms) と `Baseline` に対する増減率

対象	Baseline	Ins-only		Func-only		ARMCmp	
		時間	増減率	時間	増減率	時間	増減率
sudo	229.6	267.1	+16.3%	273.7	+19.2%	260.7	+13.5%
libpng	142.3	144.0	+1.2%	138.7	−2.5%	145.0	+1.9%
libxml2	152.9	159.3	+4.2%	155.1	+1.4%	157.6	+3.1%
libsndfile	149.8	158.0	+5.5%	144.9	−3.3%	161.8	+8.0%
libtiff	153.1	148.1	−3.3%	149.0	−2.7%	161.0	+5.2%
Lua	183.1	191.0	+4.3%	196.4	+7.3%	210.5	+15.0%
OpenSSL	153.8	165.5	+7.6%	172.9	+12.4%	179.0	+16.4%
PHP/unserialize	204.6	212.4	+3.8%	220.9	+8.0%	220.6	+7.8%
Poppler	372.3	435.3	+16.9%	441.2	+18.5%	446.5	+19.9%
SQLite3	178.7	190.8	+6.8%	167.0	−6.5%	194.9	+9.1%

表 A·3 MIPSCmp を組み込んだ ROSA および BDDM におけるバックドア検出可否

対象	ROSA		BDDM	
	Baseline	MIPSCmp	Baseline	MIPSCmp
sudo	×	✓	×	✓
libpng	×	✓	×	✓
libsndfile	×	✓	×	✓
libtiff	✓	✓	✓	✓
libxml2	×	✓	×	✓
Lua	×	✓	×	✓
OpenSSL	×	✓	×	✓
PHP/unserialize	×	✓	×	✓
Poppler	✓	✓	✓	✓
SQLite3	✓	✓	✓	✓

✓：バックドア検出，×：バックドア未検出

© 2026 Information Processing Society of Japan